

Arhitectura calculatoarelor

Probleme

~ exemple ~

Cristian Vancea

<https://users.utcluj.ro/~vcristian/AC.html>

1. Pentru a descrie un multiplexor în VHDL, cu proces între *begin* și *end* se poate folosi:

A) is; B) end; C) begin; **D) case;** E) architecture

2. Pentru a descrie un registru sincron în VHDL, cu proces, între *begin* și *end* se poate folosi:

A) if clk='1' then Q<=D; end if;

B) if clk='1' and clk'event then Q<=1 else Q<='0'; end if;

C) if clk'event and clk='1' then Q<=0; end if;

D) if clk'event and clk='1' then Q<=D; end if;

E) if clk'event and clk='1' then Q<=D; end;

3. Semnalele adiționale, necesare intern, la descrierea unei entități în VHDL se declară:

A) după *architecture* înainte de *begin*; B) în secțiunea *port*; C) după *entity*;

D) în fiecare proces; E) înainte de *architecture*

4. Un semnal în VHDL se declară folosind:

A) std_logic; B) when; C) variable; **D) signal;** E) process

5. Pentru a instanția o entitate în altă entitate se folosește:

- A) process; B) entity; C) component / port map; D) architecture;
- E) begin / end

6. Un proces în VHDL are descrierea comportamentului între:

- A) begin / end; B) if / end if; C) start / end; D) architecture / end;
- E) begin / end process

7. În etajul ID din procesorul MIPS cu ciclu unic se află:

- A) memoria; B) blocul de registre; C) ALU; D) registrul PC;
- E) memoria de instrucțiuni

8. Ordinea celor 5 etaje ale procesorului MIPS pipeline este:

- A) IF, ID, MEM, EX, WB; B) IF, EX, ID, MEM, WB; C) IF, ID, EX, MEM, WB;
- D) IF, ID, MEM, EX, WB; E) IF, EX, WB, MEM, ID

9. Instrucțiunile MIPS de tip R conțin în format:

- A) 3 adrese de registre; B) un imediat pe 16 biți;
- C) 2 adrese de registre și un imediat; D) cod de operație pe 7 biți;
- E) adresa de salt

10. Pentru instrucțiunea BEQ, la MIPS cu ciclu unic, condiția de salt este evaluată de:

A) ALU; B) PC; C) un comparator dedicat; D) memoria de instrucțiuni;
E) un multiplexor

11. Se pot scrie programe în cod-mașină pentru un procesor, dacă se știe:

A) VHDL; B) asamblare; C) arhitectura setului de instrucțiuni;
D) lista de periferice; E) numărul de registre

12. La MIPS pipeline cu 5 etaje, în secvența de instrucțiuni

ADD \$1, \$2, \$3

SUB \$3, \$4, \$5

ADD \$6, \$1, \$7

apar:

A) 0 hazarduri; B) 1 hazard RAW și 1 hazard WAR; C) 1 hazard RAW;
D) 2 hazarduri RAR; E) 2 hazarduri RAW

13. Graful de flux de date (DFG) este format din:

A) operații în noduri, cu arce care arată dependența de date; B) ASAP;
C) ALAP; D) operanzi în noduri, arcele arată operațiile; E) calea de date

14. La MIPS cu 5 etaje pot apărea următoarele hazarduri de date:

A) RAW, WAR, WAW; **B)** RAW; C) WAR; D) WAR, RAR; E) WAW, WAR

15. Dacă MIPS cu ciclu unic ar avea memorie comună de date și instrucțiuni, atunci:

A) ar fi mai rapid; **B)** nu poate funcționa pe un ciclu; C) ar fi mai încet;
D) ar elimina hazardurile; E) ar crește productivitatea

16. Notăția RTL concret este diferită de RTL abstract deoarece:

A) este mai rapidă; B) nu necesită căile de date în detaliu;
C) descompune operația abstractă în cicluri de ceas;
D) elimină erorile de programare; E) îmbunătățește securitatea

17. Valoarea -4, în complement față de 2 pe 8 biți este:

A) 00000100; **B)** 11111100; C) 11110100; D) 11111101; E) 00001100

18. Procesorul MIPS multi-ciclu diferă de MPS pipeline prin faptul că:

A) are memorii separate pentru date și instrucțiuni;
B) refolosește anumite unități funcționale; C) este mai scump;
D) se încălzește mai puternic; E) consumă mai mult curent

19. Operațiile în virgulă mobilă se implementează într-un procesor pipeline prin:

- A) circuite combinaționale puternice; B) nu sunt tratate distinct;
- C) pipeline în pipeline (în etajul EX);** D) creșterea frecvenței;
- E) rezolvarea hazardurilor

20. La începutul execuției secvenței următoare de cod toate registrele sunt nule. Precizați valoarea (în baza 10) din registrul \$5 la terminarea execuției (după ce ultima instrucțiune a părăsit pipeline-ul), pe MIPS32 pipeline **cu** unitate de înaintare (forwarding) și scriere pe frontul descendent în blocul de registre, pentru următoarea secvență de cod:

ADDI \$17, \$0, 38

$$\text{\$17} = 38$$

ADDI \$5, \$17, 36

$$\text{\$5} = 38 + 36 = 74$$

SLL \$17, \$17, 1

$$\text{\$17} = 38 \times 2 = 76$$

ADD \$5, \$5, \$17

$$\text{\$5} = 74 + 76 = 150$$

R: 150

Dacă nu avem branch-uri,
execuția pe pipeline cu
înaintare este ca pe ciclu
unic.

21. La reprezentarea numărului $X = -45$ pe 8 biți în complement față de 2 se folosește relația de transformare $X = \overline{\overline{X}}' = Y + 1$. Precizați valoarea lui Y în binar. $Y = X - 1 = -46$ în C_2 **R: 11010010**

22. Unitatea de control microprogramată a unui MIPS multi-ciclu are 15 stări și trebuie extinsă pentru a suporta o nouă instrucțiune cu 4 pași, T_0 - T_3 . Precizați adresa pasului T_3 în memoria de microinstrucțiuni după extindere.
 15 stări ocupă adr. 0-14. T_0 și T_1 sunt la comun => se mai alocă 2 locații pentru T_2 și T_3 la adresele 15, respectiv 16 => **R: 16**

20. Automatul de stări (FSM) pentru unitatea de control a unui MIPS multi-ciclu este extinsă pentru a suporta N instrucțiuni noi distincte, dintre care N-1 instrucțiuni au câte 7 pași (perioade ce ceas), iar ultima are maxim 7 pași. După extindere automatul se mărește cu 43 stări. Precizați valoarea lui N.

T_0 și T_1 sunt la comun => $(7 - 2) \times (N - 1) + Y - 2 = 43$ și $Y \leq 7$ => $N = \frac{(50 - \underset{=5}{Y})}{5}$ **R: 9**

21. Se presupune că execuția unei secvențe de instrucțiuni independente (fără hazarduri) pe un procesor MIPS pipeline cu 5 etaje echilibrate este de 19s. Care ar fi timpul de execuție pe un procesor MIPS cu ciclu unic? 19×5 => **R: 95s**

22. Transformați instrucțiunea OR \$11, \$5, \$1 în cod-mașină pentru MIPS32 știind că valoarea câmpului *function* este 10 (în baza 10).

R: 00000000101000010101100000001010

23. Precizați valoarea (în baza 10) pe intrarea de date a memoriei de date, la execuția instrucțiunii LW \$15, 33(\$10), pe un procesor MIPS32 cu ciclu unic, știind că \$10=54 și \$15=38. **Data Memoriei e legată la RD2 ⇔ RF[rt] => R: 38**

24. Precizați valoarea (în baza 10) pe intrarea de adresă a memoriei de date, la execuția instrucțiunii SLL \$7, \$13, 2, pe un procesor MIPS32 cu ciclu unic, știind că \$13=34 și \$7=13. Adresa Mem e legată la ALURes => R: $34 \times 4 = 136$

25. Pe un procesor MIPS32 cu ciclu unic locațiile din memoria de date sunt inițializate cu valori identice cu adresa lor. Precizați valoarea (în baza 10) din registrul \$1 după execuția secvenței următoare de cod:

ADDI \$17, \$0, 14 \$17 = 14

SLL \$17, \$17, 2 \$17 = $14 \times 4 = 56$

LW \$1, 0(\$17) \$1 = Mem[56] = 56

SLL \$17, \$17, 1 \$17 = $56 \times 2 = 112$

LW \$17, 4(\$17) \$17 = Mem[112 + 4] = Mem[116] = 116

ADD \$1, \$1, \$17 \$1 = $56 + 116 = 172$ R: 172

26. Precizați valoarea (în baza 10) pe intrarea de adresă RA2 a blocului de registre, la execuția instrucțiunii LW \$17, -16(\$6), pe un procesor MIPS32 cu ciclu unic, știind că \$6=128 și \$17=64. Adresa RA2 e dată de câmpul rt => R: 17

27. Precizați valoarea (în baza 10) pe intrarea de date WD a blocului de registre, la execuția instrucțiunii SLL \$10, \$15, 3, pe un procesor MIPS32 cu ciclu unic, știind că \$15=25 și \$10=39. WD = ALURes = $25 \times 2^3 = 25 \times 8$ R: 200

28. Pe un procesor MIPS32 multi-ciclu registrele sunt inițializate cu valoarea 3 de la \$1 la \$7, iar de la \$8 la \$31 cu valoarea 52. Precizați valoarea (în baza 10) pe ieșirea ALU când instrucțiunea LW din următoarea secvență se află la pasul T_2 .

ADD \$6, \$15, \$6 $\$6 = 52 + 3 = 55$

SW \$6, 0(\$15) $\text{Mem}[52] = 55$

AND \$6, \$6, \$6 $\$6 = 55 \& 55 = 55$

LW \$15, 6(\$6) La T_2 , pe ALU avem adresa mem. $6 + 55 = 61$ **R: 61**

ADD \$15, \$6, \$15

29. Precizați valoarea (în baza 10) pe intrarea registrului PC, la perioada de ceas 10, pe un procesor MIPS32 multi-ciclu, dacă la perioada de ceas 0 începe să se execute secvența de instrucțiuni de la adresa 08:

08: ADDI \$13, \$0, 46 $\$13 = 46$

12: ADDI \$9, \$13, 1 $\$9 = 47$

16: BEQ \$9, \$13, 11 $C10 \Leftrightarrow T_2$ pt. BEQ: PC primește pe intrare

adresa de branch = $16 + 4 + 11 \times 4 = 64$ **R: 64**

30. La un procesor MIPS32 pipeline, cu scriere pe frontul descendent în blocul de registre și **fără** hardware de înaintare (*forwarding*), registrele sunt inițializate cu valoarea 3, exceptând registrul \$0. Precizați valoarea (în baza 10) a registrului \$6 după execuția următoarei secvențe de cod.

ADDI \$13, \$0, \$17

ADDI \$6, \$13, 45

SLL \$13, \$13, 1

ADDI \$13, \$13, 1

ADDI \$13, \$13, 1

ADD \$6, \$6, \$13

Fără forwarding + front descendent $\Leftrightarrow$ destinația se scrie după 2 instr.
deci e valabilă de la a 3-a

$$\$6 = 3 + 45 = 48$$

$$\$13 = 3 \times 2 = 6$$

$$\$6 = 48 + 6 = 54 \quad \text{R: 54}$$

31. Un predictor bimodal cu 2 biți, care folosește numărător cu saturație, are 4 stări de la 0 (Strongly Not Taken) la 3 (Strongly Taken). Valoarea curentă a predictorului este 1_{10} . Precizați care va fi valoarea acestuia (în baza 10) după următoarele 4 salturi, dacă rezoluția acestora (în ordine cronologică) este: NT, NT, T, T (T = Taken, NT = Not Taken). $1 - 1 = 0 - 1 = 0 + 1 = 1 + 1 = 2 \quad \text{R: 2}$

32. Un tabel conține 32 predictorii bimodali cu saturație, pe 2 biți. Precizați (în baza 10) indexul predictorului selectat pentru instrucțiunea BEQ \$5, \$3, -3, de la adresa 24 din memoria de instrucțiuni, dacă \$5=1, \$3=1 și instrucțiunea are 4 octeți. Biții $PC_{1:0}$ se ignoră la indexare $\Rightarrow (24 / 4) \% 32 = 6 \quad \text{R: 6}$

33. Precizați capacitatea unei memorii cache mapată direct (*Direct Mapped*), care are blocuri de 4 semi-cuvinte a câte 2 Bytes, știind că procesorul generează adrese pe 32 de biți și dimensiunea câmpului index este 10 biți.

$$2^{10} \text{ blocks} \times 4 \text{ halfwords} \times 2\text{B} = 8\text{KB} \quad \text{R: } 8\text{KB}$$

34. Precizați capacitatea unei memorii cache asociativă pe set cu 8 căi (*8-way Set Associative*), care are blocuri de 16 Bytes, știind că procesorul generează adrese pe 32 de biți și dimensiunea câmpului *Tag* este 15 biți.

$$\text{Index} = 32 - 15 (\text{tag}) - 4 (\text{offset}) = 13 \text{ biți} \Rightarrow 2^{13} \text{ blocks}$$

$$\text{Dimensiune cache} = 2^{13} \text{ blocks} \times 8 (\text{ways}) \times 16\text{B} = 2^{20}\text{B} \quad \text{R: } 1\text{MB}$$

35. Precizați dimensiunea câmpului *Tag* la o memorie cache asociativă pe set cu 2 căi (*2-way Set Associative*), care are blocuri de 4 Bytes, știind că procesorul generează adrese pe 32 de biți și capacitatea memoriei (strict zona de date) este 256KB.

$$\text{Nr. blocuri} = 256\text{KB} / 4\text{B} = 64\text{K} \Rightarrow \text{Nr. seturi} = 64\text{K} / 2 = 32\text{K} = 2^{15} \Rightarrow$$

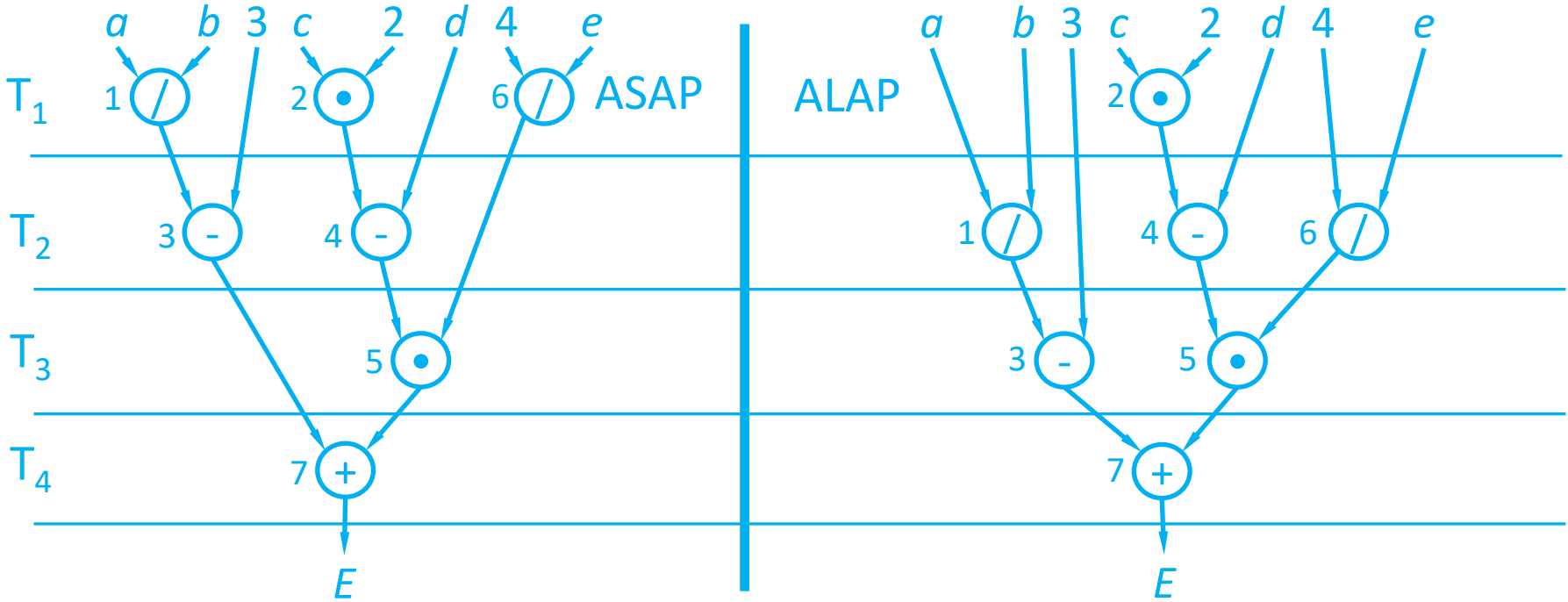
$$\text{Dimensiune Index} = 15 \text{ biți}$$

$$\text{Dimensiune bloc} = 4\text{B} = 2^2 \Rightarrow \text{Dimensiune Offset} = 2 \quad \left| \Rightarrow \text{R: } 32 - 15 - 2 = 15 \text{ biți} \right.$$

36. Precizați dimensiunea câmpului *Index* la o memorie cache asociativă pe set cu 8 căi (*8-way Set Associative*), care are blocuri de 16 Bytes, știind că procesorul generează adrese pe 32 de biți și câmpul *Tag* are 20 biți.

Dim. bloc = 16B = 2^4 B => Offset = 4 biți => Index = 32 - 20 - 4 = 8 biți => **R: 8 biți**

37. Se consideră expresia $E = a / b - 3 + (c \cdot 2 - d) \cdot 4 / e$, care va fi implementată în forma dată, **fără alte modificări**. Desenați DFG-urile rezultate la planificarea fără constrângeri de resurse în cadrul sintezei de nivel înalt (HLS), în variantele ASAP (As Soon As Possible) și ALAP (As Late As Possible). Precizați mobilitatea maximă întâlnită și la care din operații apare.



Mobilitate 1 la operațiile 1, 3, 6

38. Se consideră variabilele de tip întreg x și y în memoria de date la adresele absolute A_1 și A_2 . Scrieți o secvență de program pentru MIPS32 cu ciclu unic, echivalentă secvenței de cod C următoare. La finalul executiei, valoarea variabilei z se salvează în memoria de date la adresa absolută A_3 . **Folosiți valori numerice direct în codul de asamblare, propuse individual, pentru A_1 , A_2 , A_3 și valori numerice pentru adresele de salt utilizate în program, specificate clar în răspuns.** Adresa primei instrucțiuni se consideră 0. Explicați foarte sumar codul, prin comentarii în dreptul liniilor relevante (câteva cuvinte, roluri relevante pentru registre, etc.)

```
int z = 4;
```

```
z = z + x;
```

```
if (x == 0) then {
```

```
    z = z - y;
```

```
}
```

Adresele în memorie trebuie să
fie multiplu de 4!

39. Reproiectați/extindeți procesorul MIPS32 cu ciclu unic pentru a suporta și instrucțiunea **SUMM \$rt, \$rs, imm** cu funcționalitatea dată de următoarea descriere RTL: $RF[rt] \leftarrow Mem[RF[rs]] + S_Ext(offset)$, $PC \leftarrow PC + 4$. Respectați și parcurgeți pașii de proiectare cu argumente, astfel:

- Tipul de instrucțiune ales și formatul acesteia în cod-mașină, cu coduri la alegere și valori corespunzătoare pentru câmpuri, unde este cazul.
- Precizarea elementelor hardware adiționale rezultate conform analizei RTL.
- Desenarea schemei extinse, cu calea de date și semnalele de control.
original + extensii
- Definirea valorilor tuturor semnalelor de control pentru instrucțiunea adăugată. Tabel cu 2 rânduri. Pe coloane apar toate semnalele de control din procesor cu numele (sus) și una din valorile 0, 1 sau X (jos), în funcție de caz.
- Extinderea unității de control în conformitate cu modificările aduse.
Nu se desenează tot Main Control ci numai ce se adaugă pentru instrucțiune. A se analiza exemplele rezolvate la Cursul 5 în care s-a marcat ce s-a adăugat la Main Control.

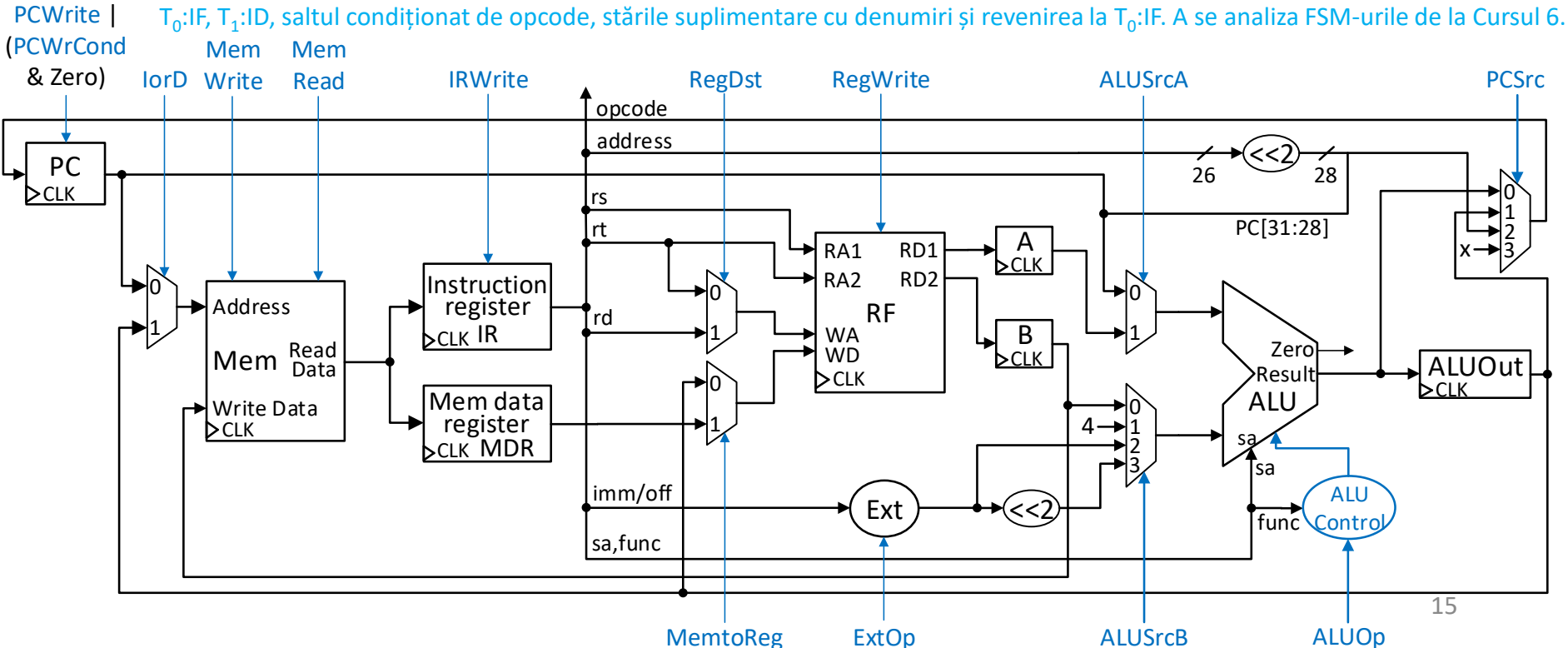
Altă variantă

- Instrucțiunea **JM \$rt, \$rs** cu descrierea RTL: $PC \leftarrow Mem[\$rs + \$rt]$.

40. Reproiectați/extindeți procesorul MIPS32 multi-ciclu pentru a suporta și instrucțiunea **SUMM \$rt, \$rs, imm** cu funcționalitatea dată de următoarea descriere RTL: **RF[rt] ← Mem[RF[rs]] + S_Ext(offset), PC ← PC + 4**. Respectați și parcurgeți cu argumente, următorii pași de rezolvare:

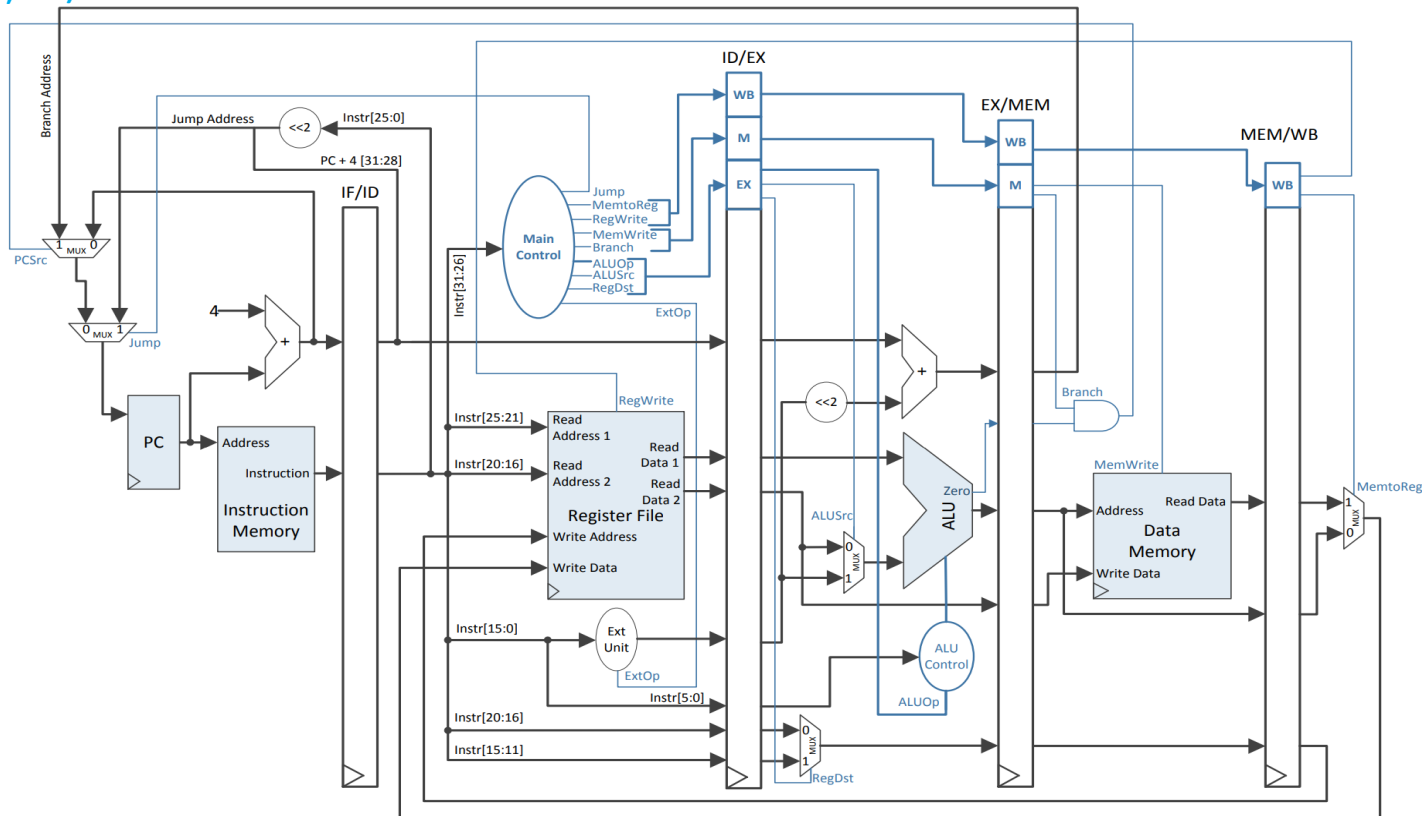
- Marcarea modificărilor necesare pe schema de mai jos sau separat.
- Descrierea RTL concret, pe pași (T_0 , T_1 , ...), pentru noua instrucțiune, cu valorile semnalelor de control relevante (diferite de X).
- Extinderea automatului unității de control FSM necesară pentru instrucțiune.

La a) se evită introducerea de UF complexe (ex. sumatoare, scăzătoare, etc.) prin reutilizarea ALU, chiar dacă e necesară introducerea de noi MUX-uri, legături sau pași suplimentari în RTL concret. La c) nu se cere FSM cu toate stările! Să apară pe FSM T_0 :IF, T_1 :ID, saltul condiționat de opcode, stările suplimentare cu denumiri și revenirea la T_0 :IF. A se analiza FSM-urile de la Cursul 6.



41. Reproiectați/extindeți procesorul MIPS32 pipeline pentru a suporta și instrucțiunea **SUMM \$rt, \$rs, imm** cu funcționalitatea dată de următoarea descriere RTL: $RF[rt] \leftarrow Mem[RF[rs]] + S_Ext(offset), PC \leftarrow PC + 4$. Respectați și parcurgeți cu argumente, următorii pași de rezolvare:

- Marcarea modificărilor necesare pe schema de mai jos sau separat.
- Definirea valorilor tuturor semnalelor de control pentru instrucțiunea adăugată. c) Extinderea unității de control în conformitate cu modificările aduse. **b) Și c) au rezolvare similară cu ciclu unic.**



43. Se consideră o memorie cache total asociativă (*fully associative*), adresabilă pe octet (Byte), cu 4 blocuri de 2 cuvinte a câte 4 Bytes. Blocurile sunt numerotate începând cu 0 și pentru înlocuirea unui bloc se folosește politica FIFO. Pentru fiecare din următoarele citiri din memorie, efectuate în ordine cronologică, de la adrese exprimate în octeți (Bytes) 96, 64, 96, 0, 32, precizați și justificați următoarele:

- 1) rezoluția Hit / Miss;
- 2) blocul care se scrie în cache sau dacă nu se scrie nimic;
- 3) blocul care se înlocuiește în cache sau dacă nu se înlocuiește nimic.

Nr. bloc = $\text{adr} / (4 \times 2) \Rightarrow$ Se accesează, în ordine, blocurile: 12 8 12 0 4

R: Hit/Miss: M M H M M

Bloc scris: 12 8 - 0 4

Bloc înlocuit: - - - - -